

Relatório dos Slides

Slide 2 — Objetivo e ligação com o mundo real

O que entendi: O objetivo é entender as regras por trás da execução do código para ele fazer exatamente o que parece fazer, sem surpresas de precedência, conversão ou desvio. Para o engenheiro de software, isso serve para ler código dos outros sem ser enganado pela indentação, evitar conversões implícitas perigosas (como o `==` do JS e do PHP) e escolher a estrutura de controle mais legível.

Como eu explicaria: Entender como a linguagem avalia expressões não é só teoria, é segurança e estabilidade. Quando escrevo código, preciso ter certeza de que o computador vai interpretar do jeito que pensei.

Exemplo/Aplicação: O slide traz três casos reais de quando o fluxo "sai do controle":

- 2014, "goto fail" da Apple: uma linha goto fail; duplicada pulava a verificação da assinatura, e iOS e macOS aceitavam certificados falsos.
- 2008, tocadores Zune: no último dia de um ano bissexto, um while nunca terminava e todos os aparelhos travaram.
- PHP: `"0e4..." == "0e8..."` dá verdadeiro, então logins que comparavam hashes com `==` aceitavam senhas erradas.

Esses casos voltam nos slides 10, 19 e 20. Como eu uso PHP no dia a dia, isso reforça que devo usar sempre `===`.

Slide 3 — Roteiro da aula

O que entendi: A aula tem seis blocos: quiz de abertura (0-10 min), expressões (10-30), conversões, booleanas e curto-circuito (30-45), atribuição (45-55), seleção, iteração, goto e comandos protegidos (55-70) e o exercício "encontre o bug" (70-80). É uma aula exploratória: a turma prevê a saída do código antes de executar, e os exemplos rodam no onecompiler.com.

Slide 4 — Quiz de abertura: o que cada linha imprime?

O que entendi: Cada trecho usa uma regra que aparece depois na aula. Abaixo, as seis perguntas com resposta e motivo.

Pergunta 1 (Python):

```
print(-2 ** 2, 2 ** 3 ** 2)
```

Resposta: -4 512

Por que: O `**` tem precedência maior que o menos unário, então `-2 ** 2` é `-(2 ** 2) = -4`. E o `**` associa à direita: `2 ** 3 ** 2` é `2 ** (3 ** 2) = 2 ** 9 = 512`.

Pergunta 2 (Java):

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

Resposta: 3345

Por que: O `+` vai da esquerda para a direita. `1 + 2 = 3` (número). Ao somar com `"3"` vira texto ("`33`"), e dali em diante é só concatenação: `"334"` e depois `"3345"`.

Pergunta 3 (JavaScript):

```
console.log("5" - 2, "5" + 2);
```

Resposta: 3 52

Por que: O - só existe para números, então "5" vira 5 e dá 3. O + também serve para texto, então o JS converte o 2 em texto e concatena: "52".

Pergunta 4 (C):

```
printf("%d\n", 3 > 2 > 1);
```

Resposta: 0

Por que: O C avalia da esquerda para a direita. (3 > 2) é verdadeiro e vale 1, depois 1 > 1 é falso (0). Não funciona como na matemática.

Pergunta 5 (Python):

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

Resposta: padrão True

Por que: O 0 é falso, e o or devolve o segundo operando quando o primeiro é falso. Já 3 > 2 > 1 no Python é encadeado como na matemática: (3 > 2) and (2 > 1), que é True.

Pergunta 6 (JavaScript):

```
console.log(0 == "", "" == "0", 0 == "0");
```

Resposta: true false true

Por que: 0 == "" é true (os dois viram 0) e 0 == "0" também. Mas "" == "0" é false, porque aí as duas são texto e texto vazio é diferente de "0". Entendi que o == não é transitivo.

Slide 5 — Expressões aritméticas: questões de projeto

O que entendi: O slide lista o que quem projeta a linguagem precisa decidir: regras de precedência, de associatividade, ordem de avaliação dos operandos, restrições a efeitos colaterais, sobrecarga de operadores e mistura de tipos. Ou seja, nem toda linguagem segue a matemática da escola.

Exemplo/Aplicação: O slide usa $a + b * c$ com $a = 3$, $b = 4$ e $c = 5$. Da esquerda para a direita sem precedência dá $(3 + 4) * 5 = 35$. Da direita para a esquerda dá $3 + (4 * 5) = 23$. Com precedência (* antes de +), que é o normal, também dá 23. Ele cita ainda o APL, que sempre avalia da direita para a esquerda ($3 \times 4 + 5 = 27$, pois faz $4 + 5 = 9$ primeiro), e o Lisp, que dispensa precedência: em $(+ a (* b c))$ a notação é prefixa e os parênteses dizem a ordem.

Como eu explicaria: Se eu não conheço a precedência e a associatividade da linguagem, posso criar bugs silenciosos, que não dão erro mas dão resultado errado.

Slide 6 — Precedência e associatividade na prática

O que entendi: O slide compara o mesmo código em Python, JavaScript e Java, e os resultados mudam.

Expressão	Python	JavaScript	Java
$-2 ** 2$	-4	erro de sintaxe	não há **
$2 ** 3 ** 2$	512	512	-
$1 + 2 + "3"$	TypeError	"33"	"33"
$-7 / 2$ (inteiros)	-4 (com //)	-3.5	-3
$-7 \% 2$	1	-1	-1

Como eu explicaria: Pela tabela, o Python arredonda $-7 // 2$ para baixo (-4) e o Java trunca em direção ao zero (-3); o JS nem tem divisão inteira. No JS, $-2 ** 2$ dá erro de sintaxe: ele obriga a usar parênteses, $(-2) ** 2$ ou $-(2 ** 2)$. Sobre associatividade (quando os operadores têm a mesma prioridade): $*$ / $%$ + $-$ associam à esquerda, e o $**$ (Python, JS, Ruby, Fortran), o $++$, o $--$ e o menos unário associam à direita. No Visual Basic, o $^$ associa à esquerda.

Exemplo/Aplicação: No Excel, $=-2^2$ dá 4, porque lá o menos unário vence o $^$, o contrário do Python (-4).

Conclusão do slide: Na dúvida, usar parênteses: eles deixam a intenção clara para o compilador e para o próximo leitor.

Slide 7 — Ordem dos operandos e efeitos colaterais

O que entendi: O problema aparece quando uma expressão usa uma variável e chama uma função que altera essa mesma variável (efeito colateral). Exemplo do livro:

```
int a = 5;
int fun1(void) {
    a = 17;
    return 3;
}
int main(void) {
    a = a + fun1();
    printf("a = %d\n", a);
}
```

Como eu explicaria: O `a` começa com 5 e a `fun1` muda para 17 e retorna 3. Se o `a` for lido antes da chamada, dá $5 + 3 = 8$. Se a função for chamada primeiro, dá $17 + 3 = 20$. Segundo o slide, o Java garante a ordem da esquerda para a direita e dá 8, enquanto o C (gcc 14) e o Go dão 20, porque o padrão deles não define essa ordem.

Conclusão do slide: Sem efeitos colaterais, a ordem não importa (transparência referencial) e $f(x) + f(x)$ pode virar $2 * f(x)$. Com efeito colateral isso quebra, então é melhor evitá-los.

Slide 8 — Operadores sobrecarregados

O que entendi: Sobrecarga é o mesmo símbolo com significados diferentes: o $+$ soma inteiros, reais e concatena texto (Java). O risco é a confusão, como o $\&$ do C++, que é "E bit a bit" e "endereço de". Quando o programador pode criar suas próprias sobrecargas, é o caso de C++, C#, Python, Rust e Kotlin. O Java não permite.

Exemplo/Aplicação (Java):

```
System.out.println('a' + 1);
System.out.println("" + 'a' + 1);
x.add(y).multiply(tres);
```

O primeiro imprime 98, porque o char `'a'` vira número e soma com 1. O segundo imprime `a1`, porque começa com texto e vira concatenação. O terceiro mostra que o `BigDecimal` usa métodos (`add`, `multiply`) em vez de $+$ e $*$, já que o Java não deixa criar sobrecarga.

Exemplo/Aplicação (Python):

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, outro):
        return Vetor(self.x + outro.x, self.y + outro.y)
    def __mul__(self, k):
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)
print(v * 3)
```

Os métodos especiais `__add__` e `__mul__` ensinam o Python a usar `+` e `*` com objetos Vetor. A soma dá Vetor(4, 6) e `v * 3` dá Vetor(12, 18). Eu acho ótimo para a legibilidade (`v1 + v2` em vez de `v1.add(v2)`), mas entendi por que o Java proíbe: dá para esconder complexidade atrás de um símbolo simples.

Slide 9 — Conversões de tipos: alargamento, estreitamento e coerção

O que entendi: Alargamento é ir de um tipo menor para um maior (int para float), geralmente seguro. Estreitamento é o contrário e perde dados, por isso pede cast explícito. Coerção é quando o compilador converte sozinho.

Exemplo/Aplicação (Java):

```
int grande = 123456789;
float f = grande;
System.out.println((int) f);
double d = 3.99;
System.out.println((int) d);
System.out.println((byte) 200);
byte b = 10, c = 20;
byte s = b + c;
System.out.println(1 / 2);
System.out.println(1 / 2.0);
```

Esse exemplo mostra: o int 123456789 vira float e, ao voltar, é 123456792, ou seja, o Java mudou o número sem avisar; (int) 3.99 trunca para 3; (byte) 200 dá -56, porque sobram só os 8 bits de baixo; byte s = b + c dá erro de compilação, porque b + c é int e não cabe de volta num byte sem cast; e 1 / 2 dá 0 (divisão inteira), enquanto 1 / 2.0 dá 0.5.

Linguagem	int para real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	um só tipo int
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F#	não	erro de compilação

Como eu explicaria: C e Java convertem sozinhos; Go, Rust, ML e F# se recusam a compilar sem conversão explícita. Eu prefiro o jeito do Rust e do Go, porque o caso do 123456789 mostra que a coerção pode estragar o valor sem eu perceber. O slide avisa que a coerção esconde erros: em `d = b * a`, com a digitado no lugar de c, o Java converte e compila.

Slide 10 — Coerção implícita: JavaScript e PHP

O que entendi: O slide mostra as esquisitices do "type juggling": o interpretador tenta adivinhar o que eu quero. Resultados em JavaScript:

Expressão	Resultado	Expressão	Resultado
"5" - 2	3	undefined + 1	NaN
"5" + 2	"52"	"b" + "a" + +"a" + "a"	"baNaN"
"5" * "2"	10	0 == ""	true
true + 1	2	"" == "0"	false
[] + {}	"[object Object]"	0 == "0"	true
null + 1	1	0 === "0"	false

Entendi que o - força número e o + prefere texto. O true vira 1, o null vira 0 e o undefined vira NaN. O "baNaN" acontece porque "+" tenta virar número, dá NaN, e o NaN é concatenado como texto entre "ba" e "a". O == não é transitivo, e o === não converte nada.

Exemplo/Aplicação (PHP):

```
var_dump("0e123" == "0e456");
var_dump("0e123" === "0e456");
var_dump("1" == "01");
md5("240610708");
md5("QNKCDZO");
```

O PHP acha que "0e123" é notação científica (0 vezes 10 elevado a 123), então vira 0, e "0e456" também. Por isso o primeiro dá true e o segundo (===) dá false. Os dois hashes MD5 do exemplo começam com "0e" seguido só de números, então um login que compara hashes com == aceita a senha errada. É o caso do slide 2.

Conclusão do slide: Em JS e PHP, sempre comparar com === e !==. Como trabalho com essas linguagens, isso é bem real para mim.

Slide 11 — Expressões relacionais e booleanas

O que entendi: Cada linguagem decide o que é "verdadeiro". No C não existe tipo booleano: 0 é falso e qualquer outro valor é verdadeiro.

Exemplo/Aplicação (C):

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);
printf("%d\n", (a > b) && (b > c));
if (-1) printf("verdadeiro\n");
```

O $a > b > c$ dá 0, porque $(3 > 2)$ vale 1 e $1 > 1$ é falso. Para dizer o que eu quero, preciso de $(a > b) \&\& (b > c)$, que dá 1. E o if (-1) entra, porque -1 é diferente de 0.

Exemplo/Aplicação (Python):

```
a, b, c = 3, 2, 1
print(a > b > c)
```

Imprime True, porque o Python encadeia como $(a > b)$ and $(b > c)$. Contam como falso em um if: 0, 0.0, "", [], {} e None. Contam como verdadeiro: "0", [0] e " ", porque não estão vazios.

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Entendi que o if flexível é prático, mas o if que exige booleano evita erros.

Slide 12 — Avaliação em curto-circuito

O que entendi: A linguagem para de avaliar assim que o resultado está garantido. Num E (&&) com o primeiro lado falso, o resultado já é falso. Num OU (||) com o primeiro lado verdadeiro, já é verdadeiro.

Exemplo/Aplicação (Java):

```
while (i < lista.length && lista[i] != chave)
    i++;
String s = null;
s != null && s.length() > 0
s != null & s.length() > 0
```

No while, o && evita acessar lista[lista.length]: quando o i chega ao tamanho da lista, a primeira condição vira falsa e o acesso nem acontece. Com s nulo, a primeira expressão (&&) dá false sem chamar s.length(), mas a segunda (&, que avalia os dois lados) dá NullPointerException.

Exemplo/Aplicação (JavaScript):

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade
const qtd = 0;
qtd || 10
qtd ?? 10
```

O `?.` para no primeiro null e devolve undefined, sem erro. O `qtd || 10` dá 10, porque o 0 conta como falso e se perdeu. O `qtd ?? 10` dá 0, porque só troca null e undefined.

Cuidado: Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. E no Python, `0` or "padrão" devolve "padrão", porque and e or devolvem um dos operandos, não necessariamente True ou False.

Slide 13 — Sentenças de atribuição

O que entendi: Em C, a atribuição é uma expressão: devolve o valor atribuído. Em linguagens mais rígidas, é só uma sentença.

Exemplo/Aplicação (C):

```
sum = count = 0;
a = b + (c = d / b) - 1;
while ((ch = getchar()) != EOF) { }
if (x = y) { }
```

Como a atribuição devolve valor, dá para zerar os dois com `sum = count = 0`, atribuir no meio de uma conta (exemplo do livro) e atribuir dentro do while (lê, guarda no ch e já compara com EOF). O lado ruim é o `if (x = y)`: eu queria comparar, mas ele atribui y a x e testa o valor, e compila. Já fiz esse erro por acidente.

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Exemplo/Aplicação (Python):

```
a, b = b, a
if (n := len(dados)) > 5: ...
```

A atribuição múltipla troca dois valores sem variável temporária. O operador morsa (`:=`, Python 3.8+) calcula, guarda em n e testa na mesma linha, com um símbolo diferente do `=` para evitar confusão. Nas linguagens funcionais (val no ML, let no F# e no Rust), a atribuição só dá nome a um valor que não muda.

Slide 14 — Atribuição de modo misto

O que entendi: Quando os tipos dos dois lados do `=` são diferentes, a linguagem decide se aceita, converte ou dá erro.

Exemplo/Aplicação (Java, só aceita alargamento):

```
double d = 3;
int i = 3.7;
int i = (int) 3.7;
long l = i;
i += 3.7;
System.out.println(i);
```

int para double funciona (alargamento); `int i = 3.7` é erro (estreitamento), e com `(int)` vira 3. A parte perigosa é o `i += 3.7`, que compila porque o `+=` tem um cast escondido: `i = (int) (i + 3.7)`. Com `i = 3`, vira

(int) 6.7 = 6. Já o C converte em silêncio em qualquer direção: int i = 3.7 vira 3, char c = 300 vira 44 e unsigned int u = -1 vira 4294967295.

Armadilha clássica:

```
double taxa = acertos / total * 100;
```

Com inteiros, dá 0.0, porque a divisão inteira acontece antes do alargamento para double. O tipo da variável de destino não muda a conta. Resolvido no exercício 6.

Slide 15 — Estruturas de controle: quantas são necessárias?

O que entendi: As três básicas são sequência (uma sentença após a outra), seleção (if) e iteração (while). Böhm e Jacopini provaram em 1966 que elas bastam para qualquer algoritmo, e que o goto não é necessário.

Como eu explicaria: For, switch, do e foreach existem por facilidade de escrita, não dão poder novo. O limite é a legibilidade: quanto maior a linguagem, menos do código dos outros eu reconheço.

Slide 16 — Seleção de dois caminhos: o "else pendente"

O que entendi: É a dúvida de qual if o else pertence quando há ifs aninhados sem chaves. Exemplo do livro (C):

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;
printf("%d\n", result);
```

Como eu explicaria: A endentação sugere que o else é do if de fora, mas em Java, C, C++ e C# ela é ignorada, e o else pertence ao if mais próximo que ainda não tem else. Aqui sum == 0 é verdadeiro, count == 0 é falso, então cai no else do if interno e imprime 1. Se eu quisesse o else do if de fora, precisaria de chaves em volta do interno (aí imprimiria -1). Só Python e F# levam a endentação em conta; Perl, Go e Rust exigem chaves sempre, o que eu prefiro.

Seletores que devolvem valor:

```
let y = if x > 0 { x } else { 2 * x };
y = x if x > 0 else 2 * x
```

O primeiro é Rust, em que o if é uma expressão e dá para atribuir direto. O segundo é a expressão condicional do Python.

Slide 17 — Seleção múltipla: do switch de C ao match

O que entendi: O slide compara o switch antigo do C com versões mais seguras.

Exemplo/Aplicação (C, sem break):

```
switch (index) {
    case 1:
    case 3: odd += 1;
    case 2:
    case 4: even += 1;
    default: printf("Error\n");
}
```

Com index = 1, entra no case 1, cai no case 3 (odd += 1), e como não tem break continua caindo: even += 1 e ainda imprime "Error". Ou seja, odd, even e o erro ao mesmo tempo. Com break em cada grupo, só o certo executaria.

Exemplo/Aplicação (Java 14+ e Rust):

```
String tipo = switch (dia) {
    case SAB, DOM -> "fim de semana";
    case SEG, TER, QUA, QUI, SEX -> "dia útil";
};

match n {
    i32::MIN..-1 => "negativo",
    0 => "zero",
    1..=9 => "um dígito",
    _ => "vários dígitos",
}
```

No Java, o -> dispensa break, não cai nos outros casos, devolve o valor direto para a variável e, em enum, é exaustivo. No Rust, o match exige cobrir todas as possibilidades: sem o _ o código não compila. No Go, cada caso termina sozinho e cair exige fallthrough; no C#, cair implicitamente é proibido; no Python 3.10 existe match com padrões e guardas.

Slide 18 — Laços controlados por contador

O que entendi: O for do C é um laço lógico disfarçado: dá para alterar a variável do laço dentro do corpo. O for do Python percorre um iterável.

Exemplo/Aplicação (C):

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
    i += 2;
}
for (; i <= 100; )
    soma += i++;
```

A saída é 0 3 6 9: imprime 0, o i += 2 e o i++ levam a 3; imprime 3 e vai a 6; imprime 6 e vai a 9; imprime 9, vai a 12 e a condição falha. O segundo for mostra que as três partes são opcionais.

Exemplo/Aplicação (Python):

```
for i in range(0, 10):
    print(i, end=" ")
    i += 2
for i in range(10, 0, -3):
    print(i, end=" ")
```

O Python entrega o próximo valor do range a cada volta, então o i += 2 não muda nada e imprime 0 1 2 3 4 5 6 7 8 9. O range(10, 0, -3) imprime 10 7 4 1. As questões de projeto do slide são: qual o tipo e escopo da variável, se posso alterá-la no corpo e se os parâmetros são avaliados uma vez ou a cada volta.

Slide 19 — Laços lógicos, saídas e iteradores

Exemplo/Aplicação 1 (Java, break rotulado):

```
externo:
for (int i = 0; i < m.length; i++)
    for (int j = 0; j < m[i].length; j++)
        if (m[i][j] < 0) {
            System.out.println(i + ", " + j);
            break externo;
        }
```

Um break normal só sai do laço onde está. Com o rótulo "externo", sai dos dois laços de uma vez.

Exemplo/Aplicação 2 (JavaScript):

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total);
total = 0;
for (const v of notas) total += v;
console.log(total);
```

O `for...in` percorre as chaves (índices, que viram texto), então concatena `0 + "0" + "1" + "2"` e imprime `"0012"`. O `for...of` percorre os valores e imprime `60`.

Exemplo/Aplicação 3 (bug do Zune):

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
  } else { dias -= 365; ano++; }
}
```

No último dia de um ano bissexto, `dias` vale `366`. Ele entra no `while` e no `if` do ano bissexto, mas `366 > 366` é falso e nada muda, então o laço nunca termina. Todo caminho do laço precisa fazer progresso. O `slide` também lembra que o `do-while` executa ao menos uma vez, o que é fácil de esquecer.

Slide 20 — Desvio incondicional: o goto

O que entendi: O `goto` pula para qualquer lugar do programa. Dijkstra escreveu em 1968 "Go To Statement Considered Harmful". C, C++, C# e Go têm `goto`; o Java não tem.

Exemplo/Aplicação ("goto fail" da Apple, simplificado):

```
int valida_certificado(void) {
  int err;
  if ((err = verifica_hash()) != 0)
    goto fail;
  goto fail;
  if ((err = verifica_assinatura()) != 0)
    goto fail;
fail:
  return err;
}
```

O `if` sem chaves só protege o primeiro `goto fail`; o segundo está solto e desvia sempre, então a verificação da assinatura nunca roda. O `err` continua `0`, que significa "certificado válido", e a assinatura falsa passa. A indentação enganou quem leu.

Como eu explicaria: O `goto` irrestrito deixa o fluxo difícil de seguir, mas `break`, `continue` e `return` são `gotos` restritos e melhoram a legibilidade. Com chaves obrigatórias (Go, Rust) ou `gcc -Wall` (que avisa sobre indentação enganosa), o bug teria sido pego.

Slide 21 — Comandos protegidos de Dijkstra

O que entendi: No comando protegido, as guardas (condições) decidem qual bloco executa. Exemplo do livro:

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Se `x == y`, as duas guardas são verdadeiras e qualquer ramo serve, pois o máximo é o mesmo. Todas as guardas são avaliadas; se várias são verdadeiras, a escolha é não determinística; se nenhuma é, dá erro (no `if`) ou o laço termina (no `do ... od`). Isso é a base do CSP de Hoare (1978) e do `select` do Go.

Exemplo/Aplicação (Go):

```
a <- "a"
b <- "b"
select {
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}
```

Como os dois canais estão prontos, o Go sorteia um. Repetindo 1000 vezes, o resultado foi algo como map[a:505 b:495] e muda a cada execução. Isso é bom para sistemas concorrentes justos: nenhum canal fica sempre em segundo plano.

Slide 22 — Exercício em duplas: encontre o bug (1/3)

O que entendi: Em cada trecho, o professor pede: (a) a saída, (b) a regra que causa o problema e (c) a correção.

Exercício 1 (JavaScript):

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

Resolução: A saída é "Total: 0012".

Como cheguei nisso: O for...in percorre as chaves do array, que são texto ("0", "1", "2"). O total começa em 0 e vai concatenando os índices, em vez de somar os preços.

Como corrigir: Trocar por for...of: for (const p of precos). Aí a saída é "Total: 60".

Exercício 2 (Python):

```
def desconto(preco, d=None):
    d = d or 10
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Resolução: A saída é 180.0.

Como cheguei nisso: O 0 conta como falso. Em d = d or 10, o 0 é descartado e vira 10, então a conta é 200 * 90 / 100 = 180.0. Eu queria 0% de desconto e esperava 200.0.

Como corrigir: Testar com None: if d is None: d = 10.

Slide 23 — Exercício: encontre o bug (2/3)

Exercício 3 (Java):

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

Resolução: A saída é "inválido".

Como cheguei nisso: Não tem break. Com dia = 2, entra no case 2 ("segunda"), cai no case 3 ("terça") e no default ("inválido"). Vale a última atribuição. É o problema do slide 17.

Como corrigir: Colocar break; no fim de cada case, ou usar o switch com -> do Java moderno.

Exercício 4 (C):

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Resolução: A saída é "saldo insuficiente" e "fim", mesmo com saldo de sobra (50 não é maior que 100).

Como cheguei nisso: O ponto e vírgula depois do if vira um comando vazio. O printf seguinte não faz parte do if e roda sempre. A indentação diz outra coisa, mas o compilador ignora.

Como corrigir: Tirar o ponto e vírgula. Usar chaves também deixaria a intenção clara.

Slide 24 — Exercício: encontre o bug (3/3)

Exercício 5 (Go):

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

Resolução: Imprime "dentro: 20" e "fora: 10".

Como cheguei nisso: No Go, o `:=` declara uma variável nova no escopo atual. Dentro do if, `x := x * 2` cria um novo x (20) que esconde o x de fora (shadowing). Quando o bloco acaba, ele some e o x original continua 10.

Como corrigir: Usar `=` em vez de `:=`, ou seja, `x = x * 2`. Aí o "fora" também imprime 20.

Exercício 6 (Java):

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Resolução: A saída é 0.0%.

Como cheguei nisso: acertos e total são inteiros, então `7 / 10` é divisão inteira e dá 0. Depois `0 * 100 = 0`, e só no fim vira double (0.0). O tipo double da variável não muda a conta feita antes.

Como corrigir: Fazer cast em um operando antes da divisão: `double taxa = (double) acertos / total * 100`; Aí `7.0 / 10 * 100 = 70.0`.

Slide 25 — Síntese: armadilhas e como as linguagens ajudam

O que entendi: A tabela resume a aula: cada recurso moderno existe para evitar uma armadilha antiga.

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2</code> , <code>1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>===</code>
Booleanos	0, "", [] são falsos; <code>a > b > c</code> em C	if só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	if (<code>x = y</code>)	atribuição não é expressão (Python, Go, Rust)
Seleção	switch sem break; else pendente	<code>-></code> , match exaustivo, chaves obrigatórias
Laços	for...in no JS; laço sem progresso	for...of, iteradores, break rotulado
Desvio	goto sem restrição	break, continue, return

Como eu explicaria: Mesmo com a ajuda da linguagem, cabe a mim escrever com clareza: parênteses e tipos explícitos.

Slide 26 — Atividades complementares

O que entendi: É a autopesquisa: expressões em Ruby e Lisp (§7.2.1.4 e 7.2.1.5), expressões condicionais, tabelas de desvio do switch, laços em linguagens funcionais, iteradores e blocos de Ruby, e a leitura do artigo do Dijkstra. A entrega sugerida é fazer o mesmo exemplo com switch em C, switch -> em Java, switch em Go e match em Rust, comparando o "cair" entre casos, a exaustividade e a legibilidade. A próxima aula é Subprogramas (caps. 9 e 10).

Compreensão geral da aula

O que eu entendi da aula como um todo é que toda linguagem tem regras próprias para avaliar expressões e controlar o fluxo, e que elas mudam de uma linguagem para outra. Se eu não conheço essas regras, escrevo código que parece fazer uma coisa e faz outra.

Em expressões, vi que precedência e associatividade decidem a ordem das contas (-2 ** 2 dá -4 no Python e 4 no Excel), que a ordem dos operandos importa com efeito colateral (Java dá 8, C e Go dão 20) e que a sobrecarga deixa o código expressivo mas pode esconder coisas. A coerção implícita (123456789 virando 123456792, "0e123" == "0e456" dando true) liga com os booleanos (0, "" e [] como falso) e com o curto-circuito, que protege contra nulos mas pode pular efeitos colaterais. Na atribuição, o if (x = y) e a divisão inteira antes do alargamento mostram que o tipo da variável de destino não muda a conta.

Em controle, vi que sequência, seleção e laço lógico já bastam, e o resto existe por facilidade de escrita. Os problemas vieram de brechas na sintaxe: else pendente, switch sem break, for...in, laço sem progresso (Zune) e goto duplicado (Apple). Chaves obrigatórias, match exaustivo, switch com ->, for iterável e atribuição que não é expressão nasceram para fechar essas brechas. Os seis exercícios foram bugs que dão saída errada sem erro de compilação, e cada um se encaixava numa regra da aula.

Conclusão

Depois dessa aula, consigo explicar por que o mesmo código dá resultados diferentes em linguagens diferentes e o que cada resultado do quiz significa. Também consigo achar armadilhas como == em vez de ===, switch sem break, if sem chaves ou divisão de inteiros dentro de uma conta com double, e sei corrigir. A principal lição é não confiar na intuição e deixar a intenção explícita com parênteses, chaves, comparação estrita e conversão de tipo escrita. Como trabalho com PHP e JavaScript, o cuidado com o == e a coerção é o que mais vou levar para o dia a dia.